RÉVISION BDD

Théo BIALASIK

SOMMAIRE

DICTIONNAIRE DE DONNÉES

TYPES DE DONNÉES

MCD (Modèle conceptuelle de données)

MLD (Modèle logique de données)

SCRIPT BDD

CRUD

- CREATE
- READ
- UPDATE
- DELETE

JOINTURE

INTEGRATION DANS LE BACK

REQUÊTE PRÉPARÉ

POINT SÉCURITÉ

FONCTION AGGRÉGATION

GROUP BY

DICTIONNAIRE DE DONNÉES

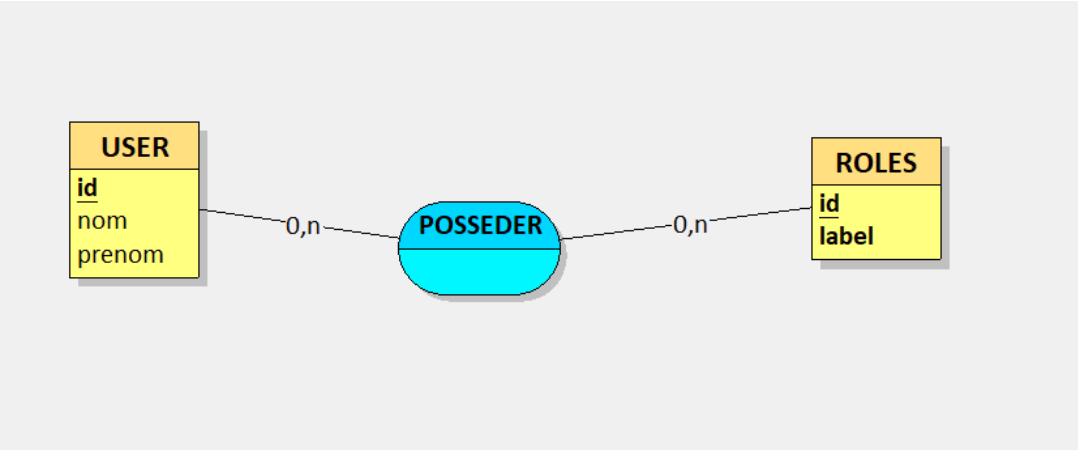
Label du champ	Type	Taille	Id potentiel	Nullable	Unique	Description
Nom Produit	ALPHANUMERIQUE	255	False	False	False	/

TYPE DE DONNÉES (POUR LE DICO)

- ALPHANUMÉRIQUE
  - Chaine de caractères : VARCHAR,TEXT,CHAR…
- NUMÉRIQUE
  - Valeur en nombre : INT, BIGINT, DECIMAL…
- LOGIQUE
  - Valeur Boolean : TRUE ou FALSE
- Temporel
  - Valeur correspondant a du temps : DATE, DATETIME, TIME…

MCD (Modèle conceptuelle de données)

- Pour trouver les cardinalités une phrase active et une phrase passive
- avec le verbe qui relie les deux tables
- Phrase active:
  - Un USER POSSEDE au MINIMUM 0 ROLES et au MAXIMUM N ROLES
- Phrase Passive
  - Un ROLE EST POSSÉDÉ au MINIMUM par 0 USER et au MAXIMUM par N USER



MLD (Modèle logique de données)

USER_ = (id *INT*, nom *VARCHAR(255)*, prenom *VARCHAR(255)*);

ROLES = (id *INT*, **label** *VARCHAR(50)*);

POSSEDER = (*#FK_USER, #FK_ROLES*);

SCRIPT BDD

```sql
CREATE TABLE USER_(
  id INT,
  nom VARCHAR(255) NOT NULL,
  prenom VARCHAR(255),
  PRIMARY KEY(id)
);

CREATE TABLE ROLES(
  id INT,
  label VARCHAR(50) NOT NULL,
  PRIMARY KEY(id),
  UNIQUE(label)
);

CREATE TABLE POSSEDER(
  id INT,
  id_1 INT,
  PRIMARY KEY(id, id_1),
  FOREIGN KEY(id) REFERENCES USER_(id),
  FOREIGN KEY(id_1) REFERENCES ROLES(id)
);
```

CRUD

```sql
-- Ajouter un utilisateur
INSERT INTO USER_ (id, nom, prenom)
VALUES (1, 'Dupont', 'Jean');

-- Ajouter un rôle
INSERT INTO ROLES (id, label)
VALUES (1, 'Administrateur');

-- Associer un rôle à un utilisateur
INSERT INTO POSSEDER (id, id_1)
VALUES (1, 1);
```

CREATE (Insertion)

```sql
-- Lire tous les utilisateurs
SELECT * FROM USER_;

-- Lire tous les rôles
SELECT * FROM ROLES;

-- Lire toutes les associations utilisateur-rôle
SELECT * FROM POSSEDER;

-- Lire les rôles d'un utilisateur spécifique
SELECT R.label
FROM ROLES R
JOIN POSSEDER P ON R.id = P.id_1
WHERE P.id = 1;

-- Lire les utilisateurs ayant un rôle spécifique
SELECT U.nom, U.prenom
FROM USER_ U
JOIN POSSEDER P ON U.id = P.id
WHERE P.id_1 = 1;
```

READ (Lecture)

```sql
-- Modifier le nom d'un utilisateur
UPDATE USER_
SET nom = 'Durand'
WHERE id = 1;

-- Modifier le label d'un rôle
UPDATE ROLES
SET label = 'Super Admin'
WHERE id = 1;
```

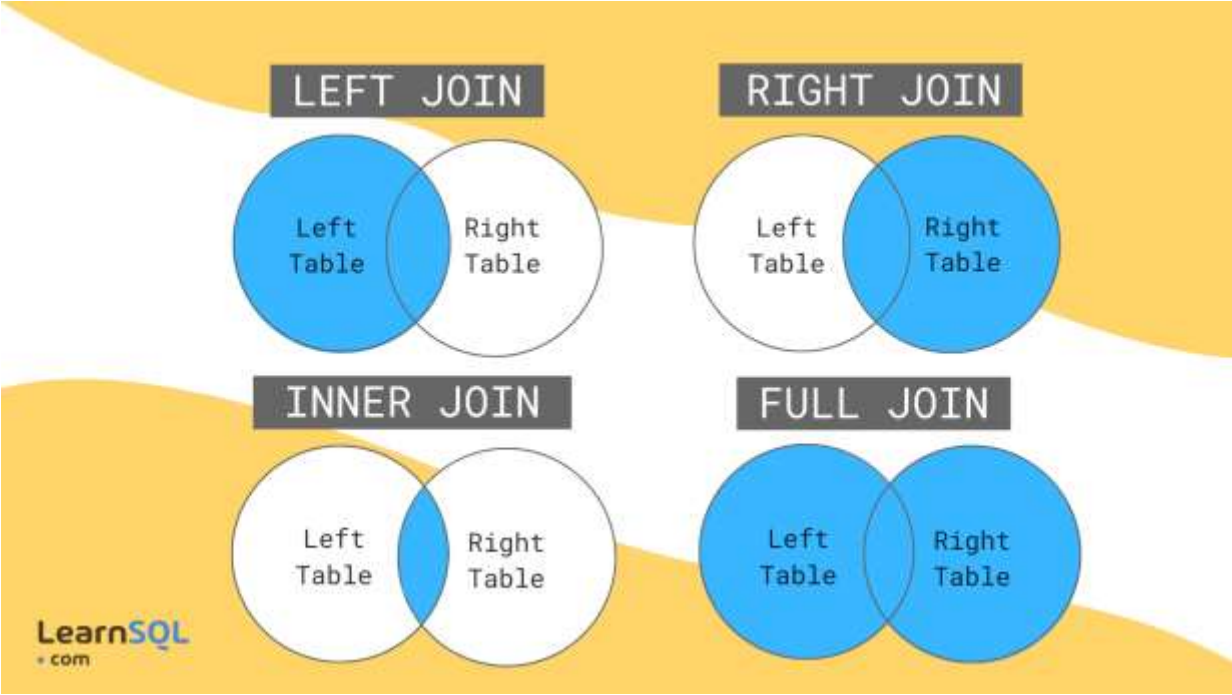
UPDATE (Modification)

```sql
-- Supprimer une association utilisateur-rôle
DELETE FROM POSSEDER
WHERE id = 1 AND id_1 = 1;

-- Supprimer un utilisateur (attention aux clés étrangères)
DELETE FROM USER_
WHERE id = 1;

-- Supprimer un rôle
DELETE FROM ROLES
```

DELETE (Suppresion)


LEFT JOIN
Left Table
Right Table
RIGHT JOIN
Left Table
Right Table
INNER JOIN
Left Table
Right Table
FULL JOIN
Left Table
Right Table
LearnSQL.com

JOITURES

```
import mysql from 'mysql2/promise';
import dotenv from 'dotenv';
dotenv.config();
const { DB_HOST, DB_USER, DB_PASSWORD, DB_NAME } = process.env;

const pool = mysql.createPool({
    host: DB_HOST,
    user: DB_USER,
    password: DB_PASSWORD,
    database: DB_NAME,
    port : DB_PORT,
});

export default pool;
```

INTEGRATION DANS LE BACK

REQUÊTES PRÉPARÉES

```
import pool from "../config/bdd.js";

const getAllUsers = async () => {
    const [rows] = await pool.query('SELECT id,nom,prenom FROM USER');
    return rows;
}

const getUserById = async (id) => {
    const [rows] = await pool.query('SELECT id,nom,prenom FROM USER WHERE id = ?', [id]);
    return rows[0];
}

const createUser = async (nom, prenom) => {
    const [result] = await pool.query('INSERT INTO USER (nom, prenom) VALUES (?, ?)', [nom, prenom]);
    return { id: result.insertId, nom, prenom };
}
const updateUser = async (id, nom, prenom) => {
    await pool.query('UPDATE USER SET nom = ?, prenom = ? WHERE id = ?', [nom, prenom, id]);
    return { id, nom, prenom };
}
const deleteUser = async (id) => {
    await pool.query('DELETE FROM USER WHERE id = ?', [id]);
    return;
}
export {
    getAllUsers,
    getUserById,
    createUser,
    updateUser,
    deleteUser
};
```

POINT SÉCURITÉ

```sql
-- 1. Créer la base de données
CREATE DATABASE IF NOT EXISTS ma_base;

-- 2. Créer l'utilisateur avec mot de passe
CREATE USER IF NOT EXISTS 'mon_user'@'%' IDENTIFIED BY 'mot_de_passe';

-- 3. Donner les permissions CRUD sur la base
GRANT SELECT, INSERT, UPDATE, DELETE ON ma_base.* TO 'mon_user'@'%';

-- 4. Appliquer les changements
FLUSH PRIVILEGES;

-- 5. Vérifier les droits (optionnel)
SHOW GRANTS FOR 'mon_user'@'%';
```

- Pensez à crée un utilisateur de base de données par projets avec des droits restreints a la database du projet avec des permission restrictive par exemple (SELECT,INSERT,DELETE,UPDATE) au moins impossible de faire de DROP DATABASE ou d’accéder aux autres database.

FONCTIONS D'AGGRÉGATION

```
USER_
+----+---------+---------+
| id | nom     | prenom  |
+----+---------+---------+
|  1 | Dupont  | Theo    |
|  2 | Dupont  | Bernard |
|  3 | Stern   | Romain  |
+----+---------+---------+
```

- MIN → Retourne la valeur minimale d'une colonne numérique.
  - Exemple :SELECT MIN(id) FROM USER_; → 1
- MAX → Retourne la valeur maximale d'une colonne numérique.
  - Exemple :SELECT MAX(id) FROM USER_; → 3
- COUNT → Retourne le nombre de lignes (ou d'éléments non NULL) dans une colonne.
  - Exemple :SELECT COUNT(id) FROM USER_; → 3
- AVG → Retourne la moyenne des valeurs numériques d'une colonne.
  - Exemple :SELECT AVG(id) FROM USER_; → 2
- SUM → Retourne la somme des valeurs numériques d'une colonne.
  - Exemple :SELECT SUM(id) FROM USER_; → 6

GROUP BY

```
SELECT nom, SUM(id) FROM USER_ GROUP BY nom;
→
+--------+----------+
| nom    | sum(id)  |
+--------+----------+
| Dupont |    3     | (1+2)
| Stern  |    3     |
+--------+----------+
```

```
SELECT nom, AVG(id) FROM USER_ GROUP BY nom;
→
+--------+----------+
| nom    | avg(id)  |
+--------+----------+
| Dupont |   1.5    | ( (1+2)/2 )
| Stern  |   3      |
+--------+----------+
```